

Fondamenti e complessità

Dal problema alla prova di difficoltà • Le frecce indicano trasformazioni o implicazioni, secondo l'etichetta.

Problema → algoritmo → costo

Definizione. Un problema specifica input e output; un'istanza assegna valori concreti ai parametri. Un algoritmo corretto termina e restituisce l'output richiesto su ogni istanza.

Specifica. Decisione: sì/no. Ricerca: una soluzione ammissibile. Ottimizzazione: una soluzione di costo minimo o massimo.

Esempio. Cammini: "esiste un cammino $s \rightarrow v$ di costo $\leq k$?" / "trovalo" / "trova quello di costo minimo".

Costo. Dimensione N: bit dell'input (costo logaritmico) oppure elementi (costo uniforme). Tempo: operazioni; spazio: celle. Dichiarare modello e caso peggiore, medio o atteso.

Attenzione. Un intero K richiede $\Theta(\log(K+1))$ bit: un costo $O(K)$ può essere esponenziale nella lunghezza della sua codifica.

P, NP e certificati

Definizione. P: problemi decisionali risolvibili in tempo polinomiale. NP: problemi decisionali con certificati positivi di lunghezza polinomiale verificabili in tempo polinomiale.

Prova. Istanza sì $\Rightarrow$ esiste un certificato accettato. Istanza no $\Rightarrow$ nessun certificato è accettato. Il verificatore riceve istanza e certificato, non deve cercare il certificato.

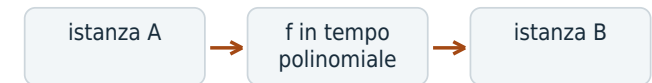
Esempio. HC: istanza = grafo; certificato = ordine dei vertici. Controlla che compaiano tutti una sola volta e che ogni coppia consecutiva, inclusa l'ultima con la prima, sia un arco.

$P \subseteq NP$ • $P = NP$ è aperto

Prova. Per P basta eseguire il risolutore ignorando il certificato. Questo non dimostra $NP \subseteq P$. SAT è NP-completo (Cook-Levin); un assegnamento è il suo certificato.

Come dimostrare NP-completezza

Definizione. Karp $A \leq_p B$: trasformazione polinomiale f con x sì per A $\Leftrightarrow f(x)$ sì per B. Conservare entrambi i versi.



B facile $\Rightarrow$ A facile

Passi. Per provare B NP-completo: 1) $B \in NP$; 2) scegli A NP-completo noto; 3) dimostra $A \leq_p B$. La transitività trasferisce la difficoltà verso B.

Definizione. NP-hard: ogni problema in NP si riduce a B; non si richiede $B \in NP$. Per ricerca/ottimizzazione precisare la riduzione con oracolo (Turing), anche con più chiamate adattive.

Teorema. Un solo NP-completo in $P \Rightarrow P = NP$. La NP-hardness è incondizionata; "nessun algoritmo polinomiale" richiede l'ipotesi $P \neq NP$.

Notazione asintotica

Definizione. Per funzioni non negative, per ogni $N \geq N_0$ e opportune costanti positive:

$$O(g): f(N) \leq c g(N)$$

$$\Omega(g): f(N) \geq c g(N)$$

$$\Theta(g): c_1 g(N) \leq f(N) \leq c_2 g(N)$$

Ricorda. O è un limite superiore, non un'uguaglianza. Un upper bound di un algoritmo dà un upper bound del problema; un lower bound del problema vale per ogni algoritmo.

Quanto cresce il costo?

Ricorda. 1, $\log \log N$, $\log N$, radici, N, $N \log N$, N^2 , N^3 , N^k : limitati da un polinomio per k costante. a^N ($a > 1$) e $N!$ sono superpolinomiali.

Nota. "Polinomiale" non significa necessariamente "un polinomio". La tesi estesa di Church-Turing motiva la stabilità della classe sui modelli classici ragionevoli.

Limite. NP-hard non significa irrisolvibile. Il problema della fermata è invece indecidibile: non ammette un algoritmo generale.

Approssimare con una garanzia

Definizione. Algoritmo polinomiale, output ammissibile; fattore $\rho \geq 1$. ALG e OPT sono i costi.

$$\text{min: } OPT \leq ALG \leq \rho OPT$$

$$\text{max: } ALG \leq OPT \leq \rho ALG$$

Prova. Per minimo: trova $LB \leq OPT$ e dimostra $ALG \leq \rho LB$. Se $OPT=0$ la garanzia impone $ALG=0$; evita rapporti con denominatore nullo.

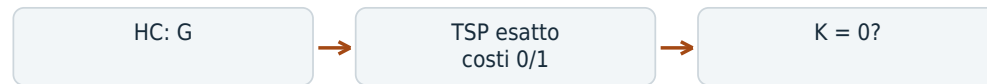
TSP generale e ricerca esatta

Grafo semplice non diretto • $n = |V| \geq 3$ • H^* = tour ottimo; cost = somma dei pesi, con molteplicità.

HC usando il TSP esatto

Specifica. $G=(V,E) \rightarrow$ sì se esiste un ciclo hamiltoniano. Usa un oracolo TSP: grafo completo pesato $\rightarrow$ tour di costo minimo.

Passi. Completa G sugli stessi vertici. Poni $c(e)=0$ per $e \in E$, $c(e)=1$ per gli archi aggiunti. Chiedi il tour ottimo di costo K ; rispondi sì $\Leftrightarrow K=0$.



$HC \leq^p TSP$ ottimizzazione

Prova. Un HC di G usa solo archi a costo 0. Viceversa, un tour a costo 0 usa solo archi di G . Quindi TSP è NP-hard.

Costo. $O(n^2)$ tempo e spazio esterni all'oracolo (matrice); una chiamata. Non è il costo del risolutore TSP.

TSP generale: nessun p costante

Ipotesi. Costi positivi; nessuna triangolare. Supponi un'approssimazione polinomiale di fattore costante $r \geq 1$.

Passi. Da HC costruisci costi 1 sugli archi originali e $r|V|+1$ sugli altri.

Prova. Caso sì: $OPT=|V| \Rightarrow ALG \leq r|V|$. Caso no: ogni tour usa un arco aggiunto $\Rightarrow ALG \geq r|V|+1+|V|-1 > r|V|$.

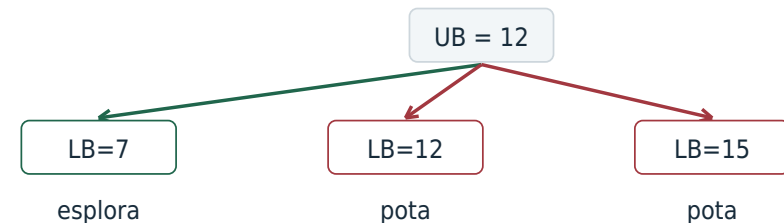
Teorema. La soglia $r|V|$ deciderebbe HC in tempo polinomiale $\Rightarrow P=NP$. Dunque nessun fattore costante, a meno che $P=NP$.

Branch & Bound: minimo esatto

FACOLTATIVO • indicazione utente

Specifica. Spazio finito di soluzioni $\rightarrow$ soluzione ammissibile di costo minimo. UB = costo della migliore già trovata; $LB(S) \leq$ costo di ogni soluzione del ramo S .

Passi. Se S è vuoto o $LB(S) \geq UB$: pota. Se S è una soluzione: valuta e aggiorna UB . Altrimenti genera figli che coprono S e richiama ricorsivamente.



Prova. Il pruning non elimina miglioramenti. I figli coprono tutte le possibilità; una misura finita deve diminuire. Esauriti i rami, UB è ottimo.

Costo. Il singolo bound può essere polinomiale, ma l'intera ricerca può essere esponenziale. Se ti fermi: $LB_glob = \min(UB, \text{bound dei rami aperti})$, $LB_glob \leq OPT \leq UB$.

TSP: lower bound con 1-tree

Passi. Nel sottografo corrente G' scegli v : MST su $V - \{v\}$, più i due archi distinti più economici incidenti a v . La somma dei costi è LB .

Prova. Ogni tour meno v è uno spanning tree; i suoi due archi su v non costano meno dei due scelti. Se il 1-tree è un tour, è ottimo nel ramo.

Passi. Altrimenti scegli w di grado ≥ 3 nel 1-tree. Per ogni arco di G' incidente a w crea un figlio che lo elimina. Ogni tour omette almeno un arco: resta in un figlio.

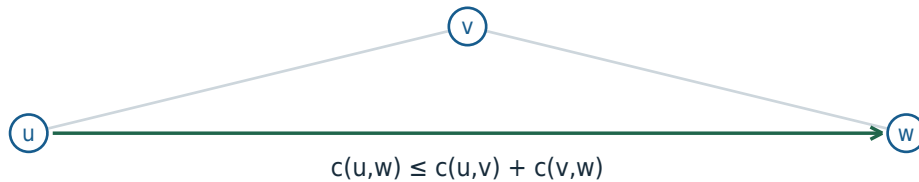
Costo. Bound e figli polinomiali per nodo; figli anche sovrapposti. Se mancano MST o due archi su v , ramo vuoto.

TSP metrico: le due approssimazioni

Grafo semplice non diretto • $n = |V| \geq 3$ • H^* = tour ottimo; cost = somma dei pesi, con molteplicità.

TSP metrico: strumenti comuni

Modello. G completo, non diretto, pesi positivi e disuguaglianza triangolare.
Output: tour ammissibile vicino all'ottimo.



Prova. Shortcut: per induzione, il costo dell'arco diretto è $\leq$ somma dei costi del cammino sostituito. Completezza $\Rightarrow$ l'arco esiste.

Ricorda. Un ciclo euleriano usa ogni arco una volta; un tour hamiltoniano ogni vertice una volta. Un multigrafo connesso con tutti i gradi pari ammette un ciclo euleriano.

Prova. MST T^* : togliendo un arco da H^* resta uno spanning tree; per minimalità, $\text{cost}(T^*) \leq \text{cost}(H^*)$.

2-approssimazione

FACOLTATIVO • non svolto AA 25/26

Passi. MST $T \rightarrow$ raddoppia gli archi $\rightarrow$ ciclo euleriano $\rightarrow$ shortcut dei vertici già visitati $\rightarrow$ tour H.

Prova. Raddoppiare rende i gradi pari e preserva la connessione. Gli shortcut non aumentano il costo.

$$\text{cost}(H) \leq 2\text{cost}(T) \leq 2\text{cost}(H^*)$$

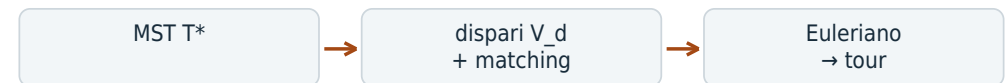
Costo. Tempo polinomiale: MST, visita e shortcut. Con matrice e Prim semplice: $O(n^2)$; memoria $O(n^2)$ includendo l'input.

Christofides: fattore 3/2

Specifica. Stesso input metrico $\rightarrow$ tour H con $\text{cost}(H) \leq (3/2)\text{cost}(H^*)$.

Passi. 1. Calcola MST T^* . 2. V_d = vertici di grado dispari in T^* . 3. Sul completo indotto G_d calcola un matching perfetto M^* di peso minimo.

Passi. 4. $G' = T^*$ più M^* , mantenendo eventuali archi paralleli. 5. Trova un ciclo euleriano. 6. Usa shortcut e chiudi il tour H.



si corregge soltanto la parità dispari

Prova. La somma dei gradi è pari $\Rightarrow |V_d|$ è pari. G_d completo $\Rightarrow$ esiste un matching perfetto. Aggiungendolo, ogni dispari diventa pari; G' resta connesso.

Prova. Da H^* salta i vertici fuori da V_d : ottieni Γ con $\text{cost}(\Gamma) \leq \text{cost}(H^*)$. Gli archi alternati di Γ formano due matching perfetti; il più economico costa $\leq \text{cost}(\Gamma)/2$.

$$\text{cost}(M^*) \leq \text{cost}(H^*)/2$$

Nota. Se $|V_d|=2$, Γ percorre due volte lo stesso arco: le due occorrenze formano i due matching.

$$\begin{aligned} \text{cost}(H) &\leq \text{cost}(T^*) + \text{cost}(M^*) \\ &\leq \text{cost}(H^*) + \text{cost}(H^*)/2 \end{aligned}$$

Costo. Tempo polinomiale, incluso il matching perfetto minimo. L'esponente dipende dall'implementazione; le dispense richiedono la polinomialità.

Attenzione. Il matching deve essere di peso minimo. Il TSP metrico resta NP-hard: costi $1/2$ (uno o due) nella riduzione HC soddisfano la triangolare; soglia n . Questo non contraddice il fattore $3/2$.

Vertex Cover: decisione, ricerca, approssimazione

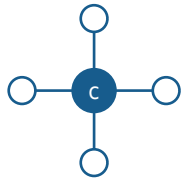
$G=(V,E)$ non diretto • $n=|V|$, $m=|E|$ • Un cover contiene almeno un estremo di ogni arco.

VC: tre scelte greedy

Specifica. $G \rightarrow$ cover C piccolo; VC cerca quello minimo. $VCD(G,k)$ chiede se esiste un cover di taglia $\leq k$.

Prova. VCD è NP-completo. Un oracolo VC dà il minimo V' ; confronta $|V'|$ con $k \Rightarrow VCD \leq^p_T VC$, quindi VC è NP-hard.

Passi. Vertice arbitrario: scegli v , aggiungilo a C , elimina gli archi incidenti. Termina quando non restano archi. Tempo $O(n+m)$, ma nessun fattore costante.



Stella: $OPT = \{c\}$

foglie: costo $n-1$

un arco: 2 estremi

Passi. Grado massimo: scegli ogni volta il vertice col maggior grado residuo. Tempo polinomiale; sistema la stella ma non garantisce un fattore costante (dispense p. 59).

Passi. Arco arbitrario: scegli (u,v) , aggiungi entrambi a C , elimina tutti gli archi incidenti. Gli archi scelti E' sono un matching massimale.



E' : archi senza estremi in comune

$OPT \geq 3$ vertici; C ne sceglie 6

Prova. Ogni cover deve usare un estremo distinto per ciascun arco di E' . Dunque $|OPT| \geq |E'|$ e $|C|=2|E'| \leq 2|OPT|$.

Costo. $O(n+m)$ tempo con liste; $O(n+m)$ spazio includendo il grafo. La stella dà rapporto 2: il bound è tight.

Relax & Round: fattore 2

Specifica. $G \rightarrow$ cover rappresentato dal vettore binario $X=APPROX$. $Cost(X)=\sum_v x_v$.

$$\min \sum_v x_v; x_u+x_v \geq 1 \quad \forall (u,v) \in E$$

Passi. ILP: $x_v \in \{0,1\}$. Rilassa a $0 \leq x_v \leq 1$ e risolvi l'LP, ottenendo X^* . Arrotonda $x_v=1$ se $x_v^* \geq 1/2$, altrimenti 0.

Prova. Su ogni arco almeno un x^* è $\geq 1/2 \Rightarrow$ l'arco è coperto. Ogni soluzione intera è ammessa nell'LP $\Rightarrow Cost(X^*) \leq Cost(OPT)$. Inoltre $x_v \leq 2x_v^*$.

$$Cost(X) \leq 2Cost(X^*) \leq 2Cost(OPT)$$

Costo. LP polinomiale; n variabili, $O(n+m)$ vincoli; rounding $O(n)$. Su ciclo pari, se l'LP restituisce tutti $1/2$, selezioni n vertici contro $n/2$ ottimi: rapporto 2.

Status. VC pesato: NON D'ESAME AA 25/26. L'estensione usa $\sum_v w_v x_v$ e lo stesso rounding.

Self-reduction con oracolo VCD

FACOLTATIVO • C pp. 56-57

Passi. Ricerca binaria su $[0,n]$ trova k ottimo. Poi, sul grafo corrente, prova $A(G-v, k-1)$. Se sì: seleziona v , rimuovilo e decrementa k . Se no: lascia il grafo invariato e passa oltre.

Prova. Resta sempre un cover compatibile col budget residuo. A fine archi, le scelte formano un cover ottimo.

Costo. $O(\log n + n)$ chiamate adattive; lavoro esterno polinomiale. Non rende VCD polinomiale.

Diffusione: modello, broadcast e wake-up

$n=|V|$, $m=|E|$; $d(u,v)$ =distanza in archi; $r(s)=\max_v d(s,v)$; $D=\max_s r(s)$. $N(x)$ =vicini di x .

Il modello viene prima del bound

Modello. Entità con stato e memoria locale; messaggi tra vicini, porte distinguibili. Ritardi finiti in assenza di guasti. Sincronia, FIFO, ID e conoscenza di n sono restrizioni aggiuntive.

Costo. M = invii, B = bit trasmessi. T_i = tempo ideale: una trasmissione per unità; T_c = massima catena causale. In asincrono non c'è un bound uniforme sul tempo fisico senza limiti ai ritardi.

Ipotesi. In questa pagina: BL = link bidirezionali, CN = grafo connesso, TR = nessun guasto. Un iniziatore, tranne WFlood. Ogni nodo conosce i propri vicini.

WFlood / Wake-up

Specifica. Almeno un risveglio spontaneo $\rightarrow$ tutti AWAKE. Sveglia spontaneamente: invia W a tutti. Sveglia su W : inoltra tranne al mittente. Dopo, ignora ogni W .

Prova. Stessa frontiera del Flooding. k conta chi si sveglia spontaneamente prima di ricevere W ; gli altri $n-k$ risparmiano un invio.

$$M = 2m - (n-k), 1 \leq k \leq n$$

Tempo. Dal primo iniziatore $T_i \leq D$. Con iniziatori S simultanei: $T_i = \max_v \min_{s \in S} d(s,v)$. Terminazione locale, non rilevazione globale.

Flooding / Broadcast

Specifica. Solo s conosce $I \rightarrow$ tutti apprendono I . s invia a tutti; alla prima ricezione, inoltra a tutti tranne il mittente e passa a DONE. Ignora le copie successive.

Prova. Un nodo raggiunto deve inviare oltre ogni frontiera verso un nodo non raggiunto. Connessione e consegna finita $\Rightarrow$ tutti raggiunti. Un solo inoltra per nodo $\Rightarrow$ termina.

$$M = 2m - n + 1; T_i = r(s)$$

Limite. DONE è locale, non rileva la fine globale. $T_c \leq n-1$ fino alle prime ricezioni; $\leq n$ includendo copie ridondanti.

Ricorda. Su albero $M=n-1$. Su completo noto: simple broadcast diretto, $M=n-1$ e $T_i=1$.

Lower bound: perché è inevitabile

Ipotesi. Protocollo generico corretto su ogni grafo connesso, senza mappa globale o conoscenza di n .

Prova. Se un arco non trasporta alcun messaggio, sostituisco con un cammino attraverso un nuovo nodo dormiente. Gli estremi non distinguono la rete; il nuovo nodo non riceve nulla. Contraddizione.

$$M \geq m; \text{ Flooding usa } M < 2m$$

Tempo. Da una sorgente s servono almeno $r(s)$ unità ideali. Nel caso peggiore sulla sorgente serve D . Flooding raggiunge questi limiti.

Limite. Il bound m non vale per algoritmi che sfruttano una topologia nota: il completo noto permette $n-1$ invii diretti.

Spanning tree: parallelo o visita sequenziale?

$n=|V|$, $m=|E|$; $d(u,v)$ =distanza in archi; $r(s)=\max_v d(s,v)$; $D=\max_s r(s)$. $N(x)$ =vicini di x .

SHOUT e SHOUT+

Specifica. $G \rightarrow$ ogni nodo conosce TreeNeighbours; i due estremi concordano sugli archi dell'albero.

Passi. SHOUT: s invia Q . Alla prima Q scegli parent, rispondi YES e invia Q agli altri. Alle successive Q rispondi NO. Attendi una risposta per ogni vicino.

Modello. FIFO per la versione con NO e contatore: evita che un NO superi una Q e faccia terminare troppo presto.

Prova. Un parent scelto alla prima scoperta precede il figlio; i parent risalgono a s senza cicli. Connettività $\Rightarrow$ copertura. Gli YES identificano anche i figli.

$$\text{SHOUT: } M = 4m - 2n + 2$$

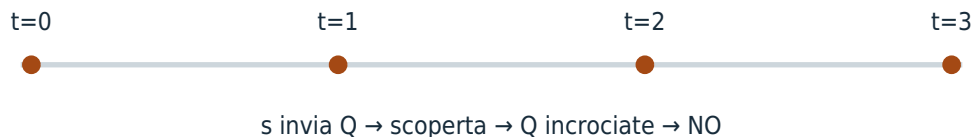
Tempo. Per $n \geq 2$, fino al DONE di tutti: $r(s) + 1 \leq T_i \leq r(s) + 2$.

Passi. SHOUT+: elimina NO; una Q incrociata conta come rifiuto implicito. Ogni arco porta Q/YES oppure Q/Q .

$$\text{SHOUT+: } M = 2m; T_i = r(s) + 1$$

Che cosa certifica la fine?

Ricorda. SHOUT/SHOUT+: ogni nodo sa di aver classificato i propri vicini, ma non rileva la fine globale. DFT: il ritorno del token consente alla radice di sapere che la visita è finita.



Esempio. Nel triangolo, SHOUT raggiunge gli altri due nodi al tempo 1; Q incrociate al tempo 2, NO al tempo 3. Per tutti DONE serve $r(s) + 2 = 3$.

Spazio. $O(\deg(x))$ record locali per vicini, archi dell'albero e stato della visita; gli identificativi, se memorizzati, hanno un costo in bit separato.

DFT: un token alla volta

Specifica. $G \rightarrow$ spanning tree di visita in profondità. Un solo token; non richiede FIFO.

Passi. ForwardToken prova un vicino non visitato. Alla prima visita memorizza parent; esplora un vicino alla volta aspettando il ritorno. ReturnToken chiude il sottoalbero; BackEdgeToken rifiuta un arco verso un visitato.

Prova. Un parent per nodo; Unvisited diminuisce. Ogni arco è classificato; quando il token torna e s esaurisce i vicini, s rileva la fine globale.

$$M = 2m; T_c = T_i = 2m$$

Ricorda. Ogni arco: Forward/Return oppure Forward/BackEdge. SHOUT può essere BFS solo con arrivi ideali per livelli; DFT può produrre grande diametro.

DFT con Visited / Ack

Passi. Alla prima visita invia Visited ai vicini tranne parent; attendi gli Ack prima di muovere il token. Chi riceve Visited rimuove il mittente dai vicini da esplorare.

Prova. Il token evita le back-edge; gli handshake con vicini diversi sono paralleli. Il token percorre soltanto i $n-1$ archi dell'albero, andata e ritorno.

$$M \leq 4m; T_i \leq 4n - 2 = O(n)$$

Costo. Tempo: $2(n-1)$ token + al più $2n$ di handshake sul cammino critico. Il termine $2n$ non conta tutti gli invii.

Limite. Più iniziatori indipendenti possono produrre una foresta. Per un albero unico: eleggi prima il leader oppure coordina costruzioni con ID.

Nota. $n=1$: nessun messaggio per tutti questi protocolli.

Saturazione ed elezione: raccogliere i valori

Leader election: esattamente un leader, tutti gli altri follower • Link affidabili, porte locali, ID distinti salvo il limite anonimo.

Saturazione su alberi

Specifica. Albero non radicato, valori locali → minimo globale. BL, CN, TR, FIFO; ogni nodo sa di essere in un albero e conosce i vicini.

Passi. 1. Wake-up. 2. Le foglie inviano il valore; un interno che ha ricevuto da tutti tranne uno aggrega e invia al vicino rimasto. 3. La coppia saturata diffonde il risultato verso l'esterno.



due parti → una coppia adiacente

Prova. Ogni messaggio riassume una componente separata dall'arco. La saturazione termina in esattamente due vicini; insieme coprono l'albero. Quale coppia emerge dipende dai ritardi.

$$M \leq 2(n-1) + n + (n-2) = 4n-4$$

Ricorda. $n \geq 2$; per $n=1$ nessun invio. Risoluzione e saturazione sono finite. Su albero radicato: broadcast dalla radice e convergecast dei risultati dai figli.

Elezione nell'albero

Passi. Radicato: eleggi la radice (0 invii; notifica $O(n)$). Non radicato: trova la coppia saturata, confronta i suoi due ID, eleggi il minore e notifica.

Costo. $M \leq 4n-4$ includendo attivazione; bit $O(n + \log \text{MaxID})$, usando $O(1)$ bit per saturare e due ID nel confronto.

Attenzione. È il minimo della coppia; eleggere il minimo globale è un'altra applicazione, che aggrega gli ID.

All The Way

Specifica. Anello uni/bidirezionale → leader con ID minimo. Più iniziatori; nessun senso globale necessario sul bidirezionale.

Passi. Ogni nodo avvia il proprio ID. Inoltra gli ID ricevuti all'altro vicino, aggiorna il minimo; ferma il proprio ID al ritorno.

Termina. Con contatore di hop, il proprio ID restituisce n . Conta gli ID ricevuti, incluso il proprio: quando sono n , hai visto tutti. Alternativa: n noto; sul modello orientato, FIFO può aiutare a riconoscere la fine.

Prova. Tutti gli ID fanno un giro completo: ciascun nodo conosce il minimo e decide il proprio stato. Nessuna notifica separata.

$$M = n^2; T_i \leq 2n-1$$

Costo. Tempo dalla prima attivazione nel caso a ritardo unitario. Gli hop richiedono anche $O(\log n)$ bit per messaggio.

Perché gli ID contano

Ipotesi. Protocollo deterministico per tutte le reti; due nodi anonimi sincroni nello stesso stato.

Prova. Ricevono gli stessi eventi e restano identici a ogni passo → entrambi leader o entrambi follower.

Limite. L'elezione non è garantita in generale senza rompere la simmetria. Una radice designata o la casualità cambiano le ipotesi.

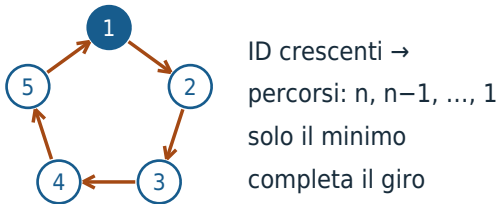
Elezione in anello: fermare i candidati

Leader election: esattamente un leader, tutti gli altri follower • Link affidabili, porte locali, ID distinti salvo il limite anonimo.

As Far As It Can / LCR

Modello. Un solo senso di inoltro; n non noto, più iniziatori.

Passi. Inoltra solo un ID minore del minimo visto. Se ti sveglia un ID maggiore del tuo, fermalo e avvia il tuo; se minore, inoltralo. Il ritorno del proprio ID elegge; poi notifica.



Prova. Il minimo non viene bloccato; ogni altro ID incontra prima un ID minore.

Costo. Worst case: $n(n+1)/2 + n$ invii. Best: $2n$ (solo il minimo iniziatore). $T_i \leq 3n-1$ con risveglio progressivo unitario e notifica.

Ricostruire il costo di LCR

Esempio. ID 1,2,3,4,5 crescenti nel senso di inoltro, tutti svegli: 1 viaggia 5 archi, 2 ne viaggia 4, poi 3,2,1. Totale 15 invii elettorali + 5 di notifica.

$$\sum_{j=1}^n j = n(n+1)/2$$

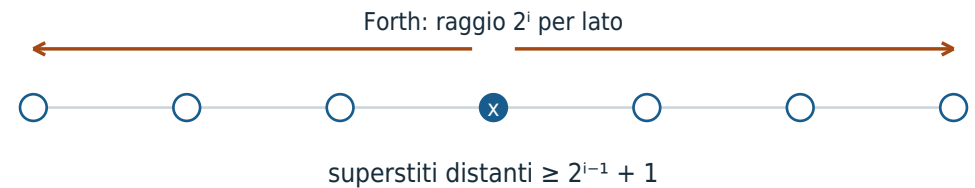
Ricorda. La somma conta tutti gli invii, anche in parallelo. Il tempo segue invece una catena: risveglio del minimo, giro dell'ID, giro di notifica.

Nota. ID minimo unico $\Rightarrow$ un solo leader. Per $n=1$ il nodo è già l'unico candidato; i conteggi a stage assumono $n \geq 2$.

Controlled Distance / HS

Modello. Anello bidirezionale, ID distinti; n non noto; più iniziatori. Candidati iniziali o attivati al risveglio.

Passi. Stage i : Forth in entrambi i sensi fino a 2^i hop; ID minore sconfigge il candidato maggiore. Se raggiunge il limite torna Back; due Back autorizzano lo stage successivo. Un Forth tornato dopo il giro elegge, poi notifica.



Prova. Il minimo globale sopravvive. Per $i \geq 1$, i candidati hanno distanza $\geq 2^{i-1} + 1$; ce ne sono al più $\lfloor n/(2^{i-1} + 1) \rfloor$. Ciascuno costa $\leq 4 \cdot 2^i$ invii: meno di $8n$ per stage interno.

$$M = O(n \log n)$$

Termina. Il raggio raddoppia: $d = \lceil \log_2 n \rceil$, stage $0, \dots, d$. Stage $0 \leq 4n$, finale $2n$, notifica n ; totale $\leq 7n + 8n(d-1)$ per $n \geq 2$.

Tempo. $T_i = O(n)$ con avvii simultanei, ritardi unitari e senza code: la somma dei raggi è geometrica. Non è un bound fisico asincrono.

Il vantaggio del raddoppio

Prova. Un raggio grande costa di più per candidato, ma lascia meno candidati: costo $O(n)$ per stage $\times O(\log n)$ stage. Il tempo somma $1 + 2 + 4 + \dots + 2^d = O(n)$.

Attenzione. Per le prove di spaziatura contano gli stage completati, anche se l'esecuzione è asincrona. Il bound temporale qui usa il modello ideale indicato.

Elezione in grafi generici

Grafi connessi, bidirezionali e affidabili; ID distinti • n = nodi, m = archi.

FloodMax su grafo generico

Modello. Link bidirezionali, FIFO, round coordinati; diametro D o un suo upper bound L noto a tutti.

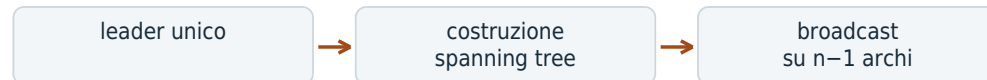
Passi. Inizia dal tuo ID; a ogni round invia il massimo noto a tutti i vicini, attendi i messaggi del round e aggiorna. Dopo L round eleggi l'ID massimo.

Invariante. Dopo r round conosci il massimo nella palla di r hop. Se $L \geq D$, tutti conoscono il massimo globale.

$$M = 2mL; T = L \text{ round}$$

Stesso problema, conoscenze diverse

Ricorda. FloodMax propaga il massimo per un numero noto di round. YO-YO propaga il minimo e riduce il grafo logico fino a poter rilevare la fine.



la notifica e la costruzione hanno costi propri

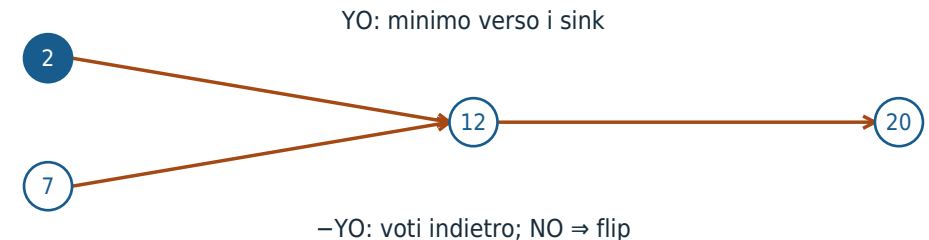
Prova. Un leader può avviare un protocollo a iniziatore unico. Se invece è già disponibile un albero con radice designata, la radice può essere leader: l'asimmetria è già data.

Nota. m conta gli archi fisici. Il pruning di YO-YO modifica gli archi logici usati, non cancella collegamenti dalla rete.

Limite. Il bound $O(m \log n)$ deriva dal lemma sulle iterazioni; la sola eliminazione di "almeno una source" darebbe soltanto $O(n)$ iterazioni.

YO-YO: senza diametro noto

Passi. Scambia ID coi vicini; orienta dal minore al maggiore: è un DAG. Source = solo uscite; sink = solo entrate.



Passi. YO: le source inviano ID; ogni interno attende tutte le entrate e inoltra il minimo. -YO: sink votano YES agli archi col minimo, NO agli altri.

Passi. Interno: se riceve un NO, restituisce NO a tutte le entrate; altrimenti YES solo da dove è arrivato il minimo. Source con NO sconfitta. Inverti gli archi con NO.

Prova. Il DAG si conserva; il minimo non è sconfitto. Collega due source se condividono un sink: in ogni cammino almeno una su due è sconfitta. I cammini superstiti dimezzano $\Rightarrow O(\log n)$ iterazioni.

Termina. Pruning: elimina sink con una sola entrata e copie duplicate del minimo, conservandone una. L'unica source resta senza archi e notifica sui link potati.

Costo. Setup $2m$, $\leq 2m$ per iterazione, notifica $n-1$: $M=O(m \log n)$. Nessun tempo preciso trasferito a ritardi asincroni.

Elezione sincrona: tempo e casualità

Grafi connessi affidabili; ID distinti per i protocolli deterministici • α = ID minimo (intero non negativo).

Speeding: ID piccoli più veloci

Modello. Anello sincrono unidirezionale; n non noto. Analisi qui con avvio simultaneo. Ogni ID occupa un messaggio logico.

Passi. Regola di eliminazione come LCR. Un ID i più piccolo viene inoltrato dopo un'attesa 2^i . Al ritorno del proprio ID: leader, poi notifica senza attesa.

Prova. Il minimo viaggia più veloce. Il candidato j -esimo ha $ID \geq \alpha + j - 1$: i suoi hop prima della notifica decrescono geometricamente. La somma resta $O(n)$.

$$M = O(n); T = O(2^n n)$$

Costo. $B = O(n \log(\text{MaxID} + 2))$. Con pacchetti di c bit bisogna contare la frammentazione: $O(n)$ invii logici non implica $O(n)$ pacchetti.

Waiting: scadenze separate

Modello. Anello sincrono orientato, n noto. Attendi $f(i, n)$; se nessuna notifica è arrivata, eleggiti e inviala.

Prova. Sia $t(i)$ il risveglio di i . Per minimo x e rivale y , serve $t(x) + f(x, n) + d(x, y) < t(y) + f(y, n)$. La notifica deve precedere la scadenza rivale.

Passi. Avvio simultaneo: $f(i, n) = ni$. Scarto tra ID consecutivi $= n >$ distanza massima $n - 1$. $M = n$ messaggi di un bit.

Passi. Risveglio progressivo: propaga wake-up e usa $f(i, n) = 2ni$. Ritardo di risveglio più distanza $< 2n$. $M = 2n$ messaggi di un bit.

$$T = O((\alpha + 1)n)$$

Nota. Il termine $+1$ copre $\alpha = 0$; non eliminare la dipendenza dal valore dell'ID.

Universal Waiting

Specifica. Grafo generico sincrono $\rightarrow$ minimo ID; n noto, tempo per arco ≤ 1 .

Passi. Diffondi start; all'attivazione attendi $2ni$. Se non arriva stop, eleggiti e diffondi stop. Chi riceve stop rinuncia e lo inoltra.

Prova. Attivazione e stop viaggiano in meno di n unità: scarto $2n$ tra scadenze consecutive evita due leader. Serve un limite sul tempo, non solo il nome "diametro".

Costo. Con due flooding: $O(m)$ invii e $O((\alpha + 1)n)$ unità; conteggio derivato sotto il modello appena dichiarato.

Leader election randomizzata

Modello. Anello anonimo sincrono orientato, n noto; round con rilevazione dei pareggi. $n \geq 2$.

Passi. A ogni round scegli 0 con probabilità $1/n$, 1 altrimenti. Waiting cerca il minimo; è leader solo se unico. Il giro in n unità lo certifica; altrimenti restart.

$$p = (1 - 1/n)^{n-1} \rightarrow 1/e$$

Prova. Successo = esattamente uno zero. I round indipendenti danno $P(\text{fallimento dopo } r) = (1 - p)^r \rightarrow 0$. Il minimo unico è un solo nodo.

Costo. $O(n)$ tempo e bit per round; $E[\text{round}] = 1/p \leq e \Rightarrow O(n)$ tempo e bit attesi. Nessun massimo deterministico di round.

Ricorda. Las Vegas: risultato corretto, qui termina con probabilità 1. Monte Carlo: tempo limitato ma possibile errore.

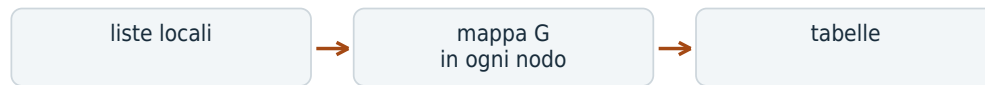
Routing: tutte le destinazioni

$n=|V|$, $m=|E|$ • Link bidirezionali, rete connessa affidabile, ID distinti; ogni nodo conosce vicini e costi locali.

Gossiping: ricostruire la mappa

Specifica. Informazioni locali → ogni nodo ottiene il grafo e calcola la tabella destinazione / next hop / costo.

Passi. Costruisci uno spanning tree (SHOUT+). Scambia le informazioni sui vicini. Ogni nodo diffonde la propria lista sull'albero; poi calcola localmente i cammini minimi.



comunicazione → calcolo locale

Prova. L'albero raggiunge tutti: ogni lista arriva a ogni nodo. Con la mappa completa, un algoritmo centrale corretto produce i cammini minimi.

Costo. Ogni elemento di vicinato percorre $n-1$ archi: $(n-1)\sum \deg(x) = 2m(n-1)$, più setup $O(m)$. Totale $O(mn)$ trasmissioni di elementi.

Spazio. $\Theta(n+m)$ record per la mappa in ciascun nodo durante la costruzione. Il calcolo locale aggiunge il costo dell'algoritmo scelto.

Attenzione. Accorpate una lista in un pacchetto cambia il numero di invii, non la quantità di dati da diffondere.

La proprietà che permette il routing

Prova. Il suffisso di un cammino minimo è ancora minimo: altrimenti sostituisco con uno più economico e migliori il cammino originale.

Spazio. Una tabella ha n record: destinazione, next hop, costo. In bit vanno contate sia la lunghezza degli ID sia quella dei costi.

Iterating / distance-vector

Specifica. G pesato con costi non negativi → distanze e next hop per tutte le destinazioni, senza mappa globale. n noto e round coordinati.

Passi. $D_0(x,x)=0$; $D_0(x,z)=\infty$ se $z \neq x$. A ogni round scambia il vettore con i vicini e aggiorna con Bellman-Ford; memorizza il vicino che realizza il minimo.

$$D_k(x,z) = \min\{D_{k-1}(x,z), \min_{y \in N(x)} [c(x,y) + D_{k-1}(y,z)]\}$$

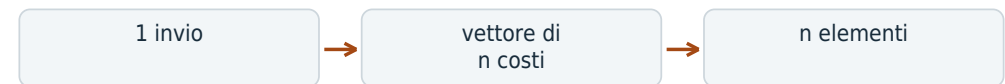
Invariante. Dopo k round conosci il minimo costo con $\leq k$ archi. Senza cicli negativi basta un cammino semplice: qui i costi non negativi garantiscono $\leq n-1$ archi.

Costo. $n-1$ round; $2m(n-1)$ invii di vettori = $O(mn)$. Ogni vettore contiene n costi $\Rightarrow O(mn^2)$ elementi trasmessi.

Spazio. n record propri; se conservi tutti i vettori ricevuti, $O(n \cdot \deg(x))$ record al nodo x . La codifica in bit dipende dai costi.

Confrontare la stessa unità

Costo. Gossiping: $O(mn)$ elementi di vicinato. Iterating: $O(mn^2)$ elementi di vettore, ma $O(mn)$ invii se il vettore intero conta come messaggio.



invii, elementi e bit sono tre misure diverse

Ricorda. Aumentare la dimensione massima di un messaggio può ridurre gli invii senza ridurre i bit. Un bound in record di memoria non è automaticamente un bound in bit.

Nota. Questi protocolli costruiscono tabelle su una topologia stabile. Cambiamenti o guasti richiedono regole ulteriori, non deducibili dal solo conteggio statico.

Routing: alberi dei cammini minimi

$n=|V|$, $m=|E|$ • Link bidirezionali, rete connessa affidabile, ID distinti; ogni nodo conosce vicini e costi locali.

Min-Hop: un livello per volta

Specifica. Sorgente s ; archi di costo uguale $\rightarrow$ albero BFS. Anche in asincrono, con livelli coordinati esplicitamente.

Passi. Start sull'albero parziale $\rightarrow$ frontiera esplora $\rightarrow$ primi arrivi scelgono parent $\rightarrow$ risposte $\rightarrow$ convergecast. Avvia il livello seguente solo quando quello attuale è chiuso.

Invariante. Inizio i : nodi a distanza $\leq i-1$. Fine i : esattamente quelli a distanza $\leq i$. Stop con n noto o dopo un livello senza nuove scoperte.

Costo. $M \leq 2(n-1)D + 2m$, più $O(n)$ per stop/verifica finale; $O(n^2)$ su grafo semplice. $T_i = O((r(s)+1)^2)$: costo $O(i)$ per livello.

Da dove vengono i due bound?

Messaggi. Sia n_i il numero di nodi nell'albero all'inizio del livello i . Start e convergecast usano $2(n_i-1)$ invii; sommando i livelli: $\leq 2(n-1)r(s)$. Le esplorazioni complessive costano $2m$.

$$M \leq 2(n-1)D + 2m + O(n)$$

Tempo. Ogni livello i : discesa $i-1$, esplorazione/risposta 2, risalita $i-1 \Rightarrow 2i$ unità ideali.

$$\sum_{i=1}^r 2i = r(r+1)$$

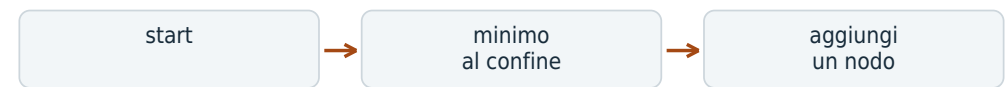
Ricorda. L'eventuale livello finale senza nuove scoperte porta a $r+1$. La garanzia di BFS dipende dalla barriera fra livelli, non dall'ordine casuale degli arrivi.



Dijkstra distribuito

Specifica. Sorgente s ; pesi strettamente positivi $\rightarrow$ albero dei cammini di costo minimo. La radice coordina ogni aggiunta.

Passi. Inizia con s e $\Delta(s)=0$. Start sull'albero; ogni nodo x offre il miglior arco (x,y) verso l'esterno secondo $\Delta(x)+c(x,y)$. Il convergecast seleziona il minimo globale.



poi aggiorna vicini e chiudi l'iterazione

Passi. Notifica il vincitore lungo il cammino dell'albero; aggiungi y con distanza definitiva. y avvisa i vicini, attende gli Ack e segnala fine iterazione a s .

Prova. Le distanze nell'albero sono definitive. Ogni cammino verso l'esterno attraversa il confine: con pesi positivi nessun cammino può battere l'offerta minima scelta.

Termina. $n-1$ aggiunte se n noto. Altrimenti convergecast con "nessun candidato" da tutti; connettività $\Rightarrow$ tutti coperti. Infine stop.

$$M = O(n^2+m); T_i = O(n^2)$$

Costo. Prima dello stop: $M \leq 2(n-1)^2 + 4m - 2(n-1)$. Su grafo semplice è $O(n^2)$. Per n sorgenti, $O(n^3)$ invii; sovrapporre le istanze non riduce gli invii.

Attenzione. Dijkstra sceglie distanza dalla sorgente; Prim sceglie peso del solo arco per un MST. Sono problemi diversi.

Una sorgente o tutte?

Costo. Su grafo semplice, ciascun albero costa $O(n^2)$ invii. Ripetere da ogni sorgente costa $O(n^3)$ invii complessivi. Il tempo dipende dall'esecuzione concorrente o sequenziale.

Guasti: collegamenti e crash sincroni

F = massimo numero di guasti ammessi, noto quando richiesto • Agreement + validità + terminazione.

Specificare cosa può guastarsi

Modello. Crash: il nodo si ferma, anche durante un invio. Omissione: un messaggio manca. Bizantino: comportamento arbitrario, anche valori diversi a destinatari diversi. Link e nodi sono componenti distinti.

Specifica. Consenso booleano: i nodi corretti decidono lo stesso bit e terminano; se tutti partono da v , decidono v . Nel modello con soli guasti di link tutti i nodi sono corretti.

Limite. Connettività per archi λ : il minimo numero di archi che disconnette. Per tollerare F omissioni permanenti occorre $F < \lambda$. Un taglio di $\leq F$ archi può separare gruppi non coordinabili.

Due generali: impossibilità

Ipotesi. Link che può perdere messaggi; protocollo deterministico che deve coordinare l'attacco, anche con clock sincroni.

Prova. Scegli un'esecuzione d'attacco con il minimo k di messaggi consegnati. $k \geq 1$: senza consegne manca il coordinamento. Elimina l'ultima consegna e ogni eventuale successiva.

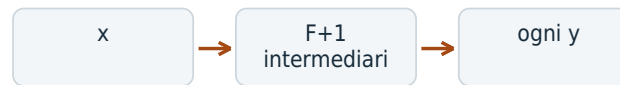
Prova. Il mittente non distingue le due esecuzioni e attacca ancora. Agreement impone che attacchi anche l'altro: ora bastano $k-1$ consegne, contraddizione.

Limite. Contare le consegne è essenziale. Conferme ulteriori spostano soltanto l'ultimo messaggio che può andare perso.

Flooding tollerante / TwoSteps

Specifica. Completo K_n ; sorgente x con $I \rightarrow$ tutti ricevono I . Nessun guasto dei nodi; $\leq F < n-1$ link con omissioni; x conosce F .

Passi. x invia a $F+1$ vicini distinti. Solo chi riceve direttamente da x inoltra a tutti gli altri, escluso x .



F guasti non bloccano $F+1$ vie disgiunte

Prova. Se y è scelto: via diretta più F vie via altri intermediari. Altrimenti: $F+1$ vie di due archi. Le vie non condividono archi $\Rightarrow$ almeno una sopravvive.

$$M \leq (F+1)(n-1); T_i \leq 2$$

Attenzione. È broadcast, non ancora consenso. $F=0$ resta $O(n)$. Su grafo generico, flooding opera nel grafo residuo connesso; il diametro rilevante è quello residuo.

Nota. Per consenso, tutti diffondono il proprio valore e applicano la stessa funzione quando la raccolta è completa. Il modello deve permettere di riconoscere il completamento. Non estendere questo argomento a corruzioni o aggiunte arbitrarie.

TellAll_Crash

Modello. Completo, link affidabili, avvio simultaneo, round sincroni, $\leq F < n$ crash; bit iniziali.

Passi. Report iniziale = proprio bit. Per $F+1$ round invia il report a tutti e sostituisco con l'AND del proprio e dei ricevuti. Messaggio assente entro fine round vale 1 (neutro). Poi decidi.



uno zero trattenuto richiede crash distinti

Prova. Se un corretto vede 0 entro F , lo invia a tutti al round seguente. Se 0 arrivasse solo ad alcuni corretti al round $F+1$, la catena precedente richiederebbe un nuovo crash a ogni round: più di F . Impossibile.

Prova. Tutti 1 $\Rightarrow$ solo 1; tutti 0 $\Rightarrow$ ciascun corretto conserva 0. L'AND non crea uno zero mai visto.

$$T = F+1 \text{ round}$$

$$M \leq n(n-1)(F+1)$$

Passi. Variante "solo zero": non inviare 1; ciascun nodo invia 0 una sola volta, appena lo conosce.

Costo. La variante usa $\leq n(n-1) = O(n^2)$ invii; servono ancora $F+1$ round. Il fattore $F+1$ copre anche $F=0$.

Consenso asincrono: quorum e casualità

Quorum = insieme di mittenti distinti • Broadcast include il mittente • I messaggi portano il numero di round.

FLP e Ben-Or con crash

Teorema. Asincrono, deterministico, anche un solo possibile crash e rete completa $\Rightarrow$ impossibile garantire consenso con terminazione in tutte le esecuzioni ammesse (FLP).

Nota. Intuizione: un ritardo non si distingue da un crash. La prova completa di FLP non è sviluppata qui; il solo timeout non risolve questa indistinguibilità.

Modello. Ben-Or: completo affidabile, crash $F < n/2$; n, F noti; bit iniziali. Attendi $n-F$ messaggi per fase; ignora round vecchi e conserva quelli futuri.

Passi. 1. Invia MyValue(r, v). Fra $n-F$ ricevuti, se $> n/2$ contengono v , invia Propose(r, v); altrimenti Propose($r, ?$).

Passi. 2. Fra $n-F$ Propose, se almeno uno è definito adotta v . Se almeno $F+1$ riportano v , decidi. Se sono tutti ?, scegli un bit uniforme per il round successivo.



$$n=7, F=2: (F+1)+(n-F) = 3+5 > 7$$

Prova. Due maggioranze strette non possono sostenere valori opposti con soli crash. Se x decide v , i suoi $F+1$ mittenti intersecano ogni quorum $n-F$: tutti adottano v . Il round seguente porta alla decisione.

Prova. Le preferenze adottate senza lancio hanno lo stesso valore. Con monete indipendenti, nel ragionamento per round del corso, la probabilità di allineamento è $\geq 2^{-(n)}$: terminazione quasi certa, $O(2^n)$ round attesi.

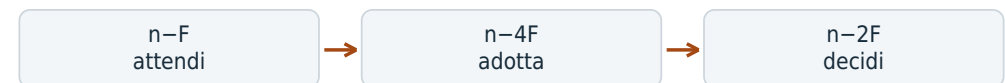
Costo. Due fasi: $O(n^2)$ invii per round. Il bound atteso conta round logici, non secondi né un massimo deterministico.

Nota. Dopo la decisione continua gli invii necessari al round seguente; un arresto immediato può impedire agli altri di completare il quorum.

Bizantino randomizzato: soglie e prova

Modello. Completo affidabile, round asincroni numerati, ID di mittente non falsificabili, bit iniziali, n, F noti, $F < n/9$. Attendi $n-F$ mittenti distinti; conserva messaggi futuri. Include il proprio messaggio.

Passi. Avvia Propose col bit iniziale. Fra $n-F$ ricevuti: se $\geq n-2F$ riportano v , adotta e decidi. Altrimenti, se $\geq n-4F$ riportano v : adotta. Altrimenti estrai un bit uniforme. Invia Propose per il round successivo prima di uscire per decisione.



le soglie riguardano lo stesso valore

Prova. Se x decide v , un altro quorum può perdere F mittenti e avere F equivoci: restano $\geq n-2F-2F = n-4F$ copie v . Tutti adottano v ; il round seguente conclude.

Prova. Due adozioni non casuali opposte richiederebbero almeno $n-5F$ corretti per valore. Anche contando F faulty: $2(n-5F)+F=2n-9F > n$, impossibile.

Prova. Tutti inizialmente v : anche sottraendo F mittenti mancanti e F faulty restano $n-2F$ corretti per v . La validità segue senza fidarsi dei bizantini.

Costo. Monete indipendenti: probabilità di allineamento $\geq 2^{-(n)}$ nel ragionamento del corso $\Rightarrow O(2^n)$ round attesi. $O(n^2)$ invii dei corretti per round; il traffico faulty va limitato a parte.

Nota. Slide p. 474: variante con $F < n/500$ e $O(n^{2.5})$ round attesi, protocollo non sviluppato. Slide p. 431: variante crash con circa $n/3$ guasti e round attesi costanti; non è il Ben-Or qui descritto.

Consenso bizantino deterministico

Quorum = insieme di mittenti distinti • Broadcast include il mittente • I messaggi portano il numero di round.

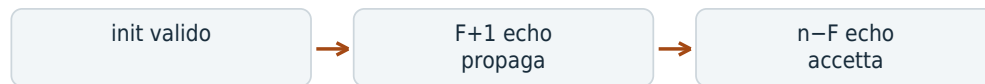
RegisteredMail: proposta coerente

STATUS D'ESAME DA CONFERMARE

Modello. Sincrono, completo, $F < n/3$ bizantini. ID noti e non falsificabili; non si assumono firme digitali. Ogni identità può originare una sola proposta.

Passi. y invia $\text{init}(0, y, t)$. Ricevuto direttamente da y al tempo $t+1$, invia $\text{echo}(0, y, t)$. Rifiuta identità, tempi e nuove origini incoerenti.

Passi. Da $t+2$: $F+1$ echo distinti $\Rightarrow$ ritrasmetti echo una volta; $n-F$ echo distinti $\Rightarrow$ accetta la proposta.



$F+1$ include almeno un corretto

Prova. Se un corretto accetta, almeno $n-2F \geq F+1$ echo sono corretti. Raggiungono tutti, che ritrasmettono; entro il round seguente tutti accettano.

Costo. Un'origine corretta è accettata in 2 unità. $O(n^2)$ invii dei corretti per proposta (ciascuno invia echo al più una volta); il traffico arbitrario richiede limiti espliciti.

Soglia deterministica bizantina

Teorema. Nel modello sincrono senza firme, con rete completa, $F \geq n/3$ impedisce il consenso deterministico. Gli ID non falsificabili non impediscono l'equivocazione del proprio mittente.

Nota. Le slide enunciano il limite; non ne ricostruiamo una prova completa. Ogni protocollo va letto con la propria soglia.

TellZero-Byz

STATUS D'ESAME DA CONFERMARE

Specifica. Stesso modello di RegisteredMail, avvio simultaneo $\rightarrow$ accordo dei corretti sul bit. Si propagano soltanto prove per 0.

Passi. $t=0$: chi parte con 0 avvia RegisteredMail. $t=2i$, $1 \leq i \leq F+1$: se non hai già originato e hai accettato $>F+i-1$ identità, avvia la tua proposta.

Passi. $t=2(F+2)$: decidi 0 se hai $\geq 2F+1$ identità accettate; altrimenti 1.

Prova. Tutti 0 $\Rightarrow$ almeno $n-F \geq 2F+1$ proposte. Tutti 1 $\Rightarrow$ solo $\leq F$ origini faulty: nessun corretto supera la prima soglia.

Prova. Una decisione 0 implica almeno $F+1$ origini corrette. Le proposte accettate si propagano; una nuova origine allo stage i porta tutti oltre $F+i$ entro lo stage $i+1$. La crescita obbliga tutti a superare la soglia finale.

$$T = 2(F+2); M = O(n^3)$$

Costo. Per contare anche i faulty: al più un messaggio per vicino per unità sincrona, come nelle slide. Nessun nodo può originare ripetutamente.

La soglia cresce con lo stage



Prova. Se un corretto origina allo stage $i \leq F$, aveva $>F+i-1$ origini accettate. Entro $2i+2$ tutti ricevono quelle e la sua: $>F+i$, abbastanza per originare allo stage successivo.

Prova. Se ci sono almeno $F+1$ origini corrette, o una nasce entro lo stage F e attiva la propagazione, oppure una nasce all'ultimo stage: le $>2F$ prove già accettate raggiungono tutti entro il tempo finale.

Chord: lookup e cambiamenti della rete

DHT = tabella hash distribuita • n = nodi presenti; b = bit degli ID (m nelle slide) • Spazio circolare $0, \dots, 2^b - 1$.

Assegnazione e lookup base

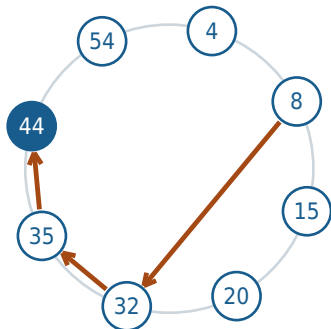
Specifica. Nodi e chiavi hanno un ID hash nello stesso spazio. Chiave $k \rightarrow$ nodo $\text{successor}(k)$, primo ID $\geq k$ in senso circolare; dopo il massimo riparti da 0.

Modello. Overlay logico; conosci il successore. Per la garanzia di base: successori corretti e nessun guasto durante la ricerca.

Passi. Se ospiti k rispondi; altrimenti inoltra al successore. La visita progredisce sull'anello fino al responsabile.

lookup base: $O(n)$ hop

ID 0 ... 63 • posizioni schematiche



con finger: $\text{lookup}(37)$, $8 \rightarrow 32 \rightarrow 35 \rightarrow 44$

Esempio. $b=6$, nodi 4,8,15,20,32,35,44,54: $k=37$ è di N44. Lo spazio ha 64 ID, ma i nodi sono soltanto 8.

Ricorda. Un hop overlay può attraversare molti link fisici. $O(\log n)$ hop non è un tempo di rete espresso in secondi.

Finger table: saltare più lontano

Passi. Al nodo i , per $j=1, \dots, b$ conserva $\text{finger}[j] = \text{successor}((i + 2^{j-1}) \bmod 2^b)$. La prima finger è il successore.

Passi. Per $\text{lookup}(k)$: controlla prima il nodo e il suo successore. Altrimenti scegli la finger che precede k più da vicino, senza superarlo. Se manca, usa il successore.

Prova. Con puntatori validi la ricerca avanza verso k senza scavalcare il responsabile. Con ID ben distribuiti, i salti riducono mediamente la distanza.

lookup: $O(\log n)$ hop medi

Spazio. b record per nodo; i bit dipendono dalla codifica di ID e indirizzi.

Limite. Il costo medio richiede distribuzione adeguata degli ID e tabelle mantenute. Non è un worst case per ogni disposizione o guasto.

Join: assegnare anche i dati

Passi. Il nuovo nodo conosce un partecipante; trova il proprio successore, inizializza predecessore e finger, aggiorna i riferimenti coinvolti e riceve le chiavi del suo intervallo.

Costo. Le slide danno $O(\log^2 n)$ tempo medio per l'aggiornamento esplicito. Il trasferimento dei valori dipende dai dati.

Prova. Inserendo x tra p e s , le chiavi nell'intervallo $(p, x]$ passano da s a x . Gli altri restano uguali.

Stabilize e refresh

Passi. A chiede a $B = \text{succ}(A)$ il predecessore B' . Se $B' \in (A, B)$, aggiorna $\text{succ}(A) = B'$. A notifica il successore scelto, che aggiorna il predecessore se opportuno.

Passi. Ricalcola periodicamente una finger. Con successori corretti e finger obsolete verso nodi vivi, il fallback permette di proseguire.

Costo. Scambi locali per chiamata; nessun bound globale di convergenza per join concorrenti arbitrari attestato dal corso.

Leave, failure e replica

Passi. Leave pulita: trasferisci dati al successore e collega predecessore e successore; poi aggiorna le finger.

Passi. Crash: prova i successori della lista di r nodi fino al primo vivo. Finger guasta: finger precedente o successor list. Stabilizza i puntatori.

Prova. Routing possibile se rimane un successore vivo raggiungibile e l'anello non è partizionato. I puntatori non recuperano i dati persi.

Passi. Replica i dati sui primi r successori. Il primo vivo diventa responsabile e ricostituisce le repliche.

Costo. $O(b+r)$ record di routing. Spazio e traffico dei dati dipendono da r e dal volume replicato.

Limite. Nessuna garanzia generale su partizioni e interleaving arbitrari: vedere coverage.md.